EPFL - Computer Science

COM-407 Advanced Networks

Lecture 01 — September 18, 2025

Author: Jonathan Dutoit
Professor: Pavlos Nikopoulos

Introduction to Internet Architecture

Whom is this course for?

Whoever wants to learn about how internet works and what is "under the hood".

Prerequisites:

- Undergraduate course in Computer Networks
- Prior experience with:
  - a programming language (e.g. Python)
  - virtual environment and virtual machines

What to expect?

No difficult concepts, but too many details

Course's plan:

- In lectures: cover the breadth of most fundamental concepts
- In labs: go in depth via carefully selected exercises (graded)

Labs

- 7 labs, posted on Moodle every 2 weeks, Wednesday evening
- mandatory and graded (∼50% of the grade)
- done on the vdi.epfl.ch (pool name: COM-407)
  - VDI saves no data, use online storage if needed, e.g. Google Drive
- Important details (all info on Moodle):
  - Each lab is split in 3-4 Lab Quizzes
  - Some labs have bonus research exercises, NOT mandatory, but interesting if you are motivated

  - Work individually or in groups, but always submit individually (to Moodle)
  - Multiple submissions allowed until deadline.

Recommendation: Start early! To avoid last time issues with virtual machines and everything.

Lecture Quizzes

- Posted on Moodle every week, after the lecture
- required but do not count for the grade
- How it works?
  - Take quizzes in sequential order
  - Try as many times as you want
  - Achieve success rate $\geq 70\%$ before submitting lab quizzes
  - All this enforced by Moodle, too

Final Exam

in January, date TBD early December

- Written, on site
- Closed book, no computer
- buy you are allowed to an exam "cheatsheet" (4xA4 pages)
- Exam questions based on both lectures and labs
- Past final exams (with solutions) available in December:
  - useful for practice, not a guideline
  - practice without looking directly at the solutions!

Recommendation: Start working for the final exam early. Reading all the solutions only is not enough!

Graded Formula

Overall grade:

$$G = \frac{FinalExam + LabGrade}{2}$$

Lab grade:

Let:

L_i = grade at lab i in scaled 1-6

$L_{avg} = \frac{L_0+...+L_5+0.5L_6}{6.5}$ (lab_6 counts with half weight, it's smaller)

$Bonus_{avg}$ = average of all bonuses (max bonus = 0.5 on scale 1-6)

Then:

$LabGrade = min(6, L_{avg} + Bonus_{avg})$

Summary

You work every week:

- Attend lecture
- Take Moodle lecture quiz (online)
- Advance / Complete lab

Start preparing for the final exam early.

Objective of this lecture

- Understand Layered Model of Computer Networks
- Know what MAC addresses, IP addresses and DNS names are

Computer Networks: A layered architecture of protocols

protocols (= set of rules and exchanged messages)

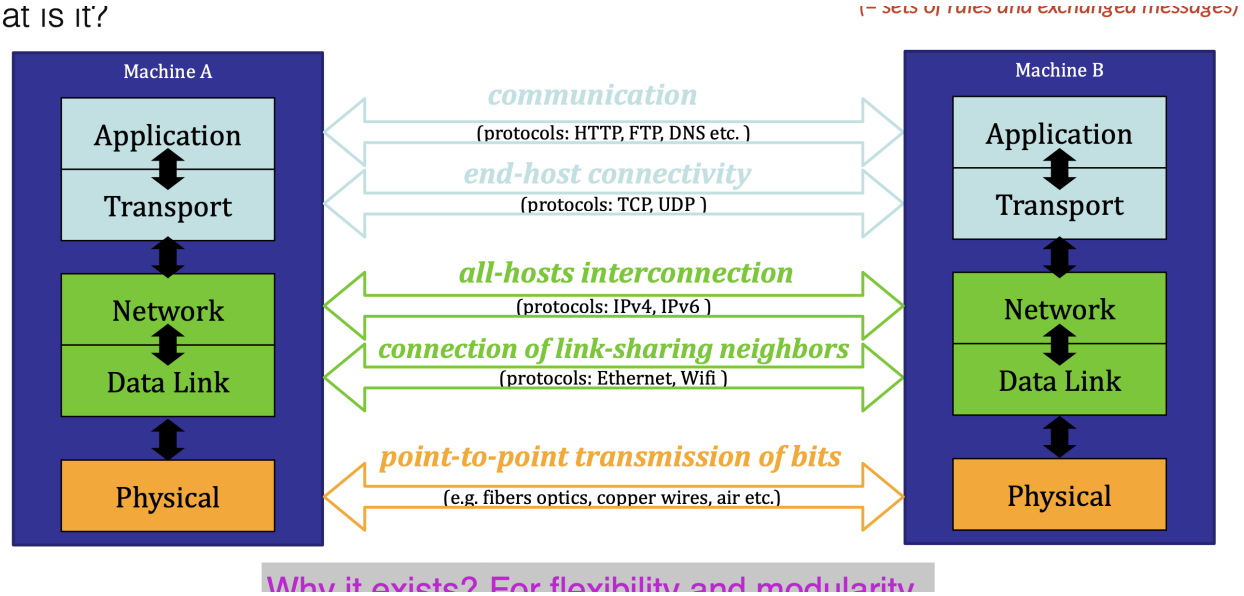


Figure 1: Computer networks diagram

- Application layer: provides network services directly to user applications.

- Transport layer: Provides end-to-end communication; can be unreliable (UDP) or reliable (TCP with acknowledgements/retransmissions).
- Network layer: Routes packets across networks with best-effort delivery (unreliable, e.g., IP).
- Data Link layer: Ensures reliable node-to-node data transfer. Transfer frames with error detection and sometimes local retransmission (hop-to-hop reliability only)
- Physical layer: Transmits raw bits over a physical medium (cables, radio, fiber)

Application Layer helps machines communicate

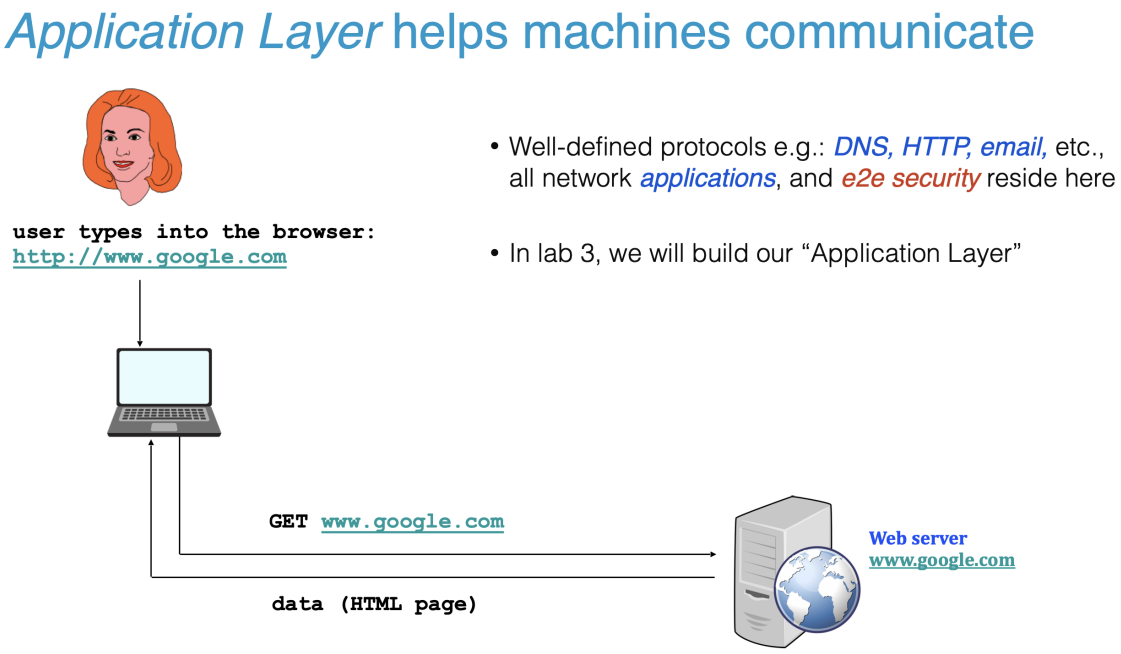


Figure 2: application layer diagram

Exercise: type curl -v www.google.com in the terminal and analyse all information.

```
❯ curl -v www.google.com
* Host www.google.com:80 was resolved.
* IPv6: 2a00:1450:400a:804::2004
* IPv4: 172.217.168.68
*   Trying [2a00:1450:400a:804::2004]:80...
*   Trying 172.217.168.68:80...
* Connected to www.google.com (172.217.168.68) port 80
* using HTTP/1.x
> GET / HTTP/1.1
> Host: www.google.com
> User-Agent: curl/8.12.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Thu, 11 Sep 2025 10:52:24 GMT
< Expires: -1
< Cache-Control: private, max-age=0
< Content-Type: text/html; charset=ISO-8859-1
< Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-
src 'nonce-O7cQUd2aG4LlTpUxPRo1rw' 'strict-dynamic' 'report-sample' 'unsafe-eval
' 'unsafe-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/oth
er-hp
< Server: gws
< X-XSS-Protection: 0
```

Figure 3: curl google.com

DNS: Domain Name System

- Why? We need a system/protocol for translating names to addresses
  - e.g.: ssc.epfl.ch ⇔ 128.178.222.83, smtp.sunrise.ch — 212.35.39.69
- names = human-friendly synonyms of IP addresses, with some structure
- What?
  - DNS clients ⇒ query servers to resolve name into an address
  - DNS servers ⇒ map host names to IP addresses and respond

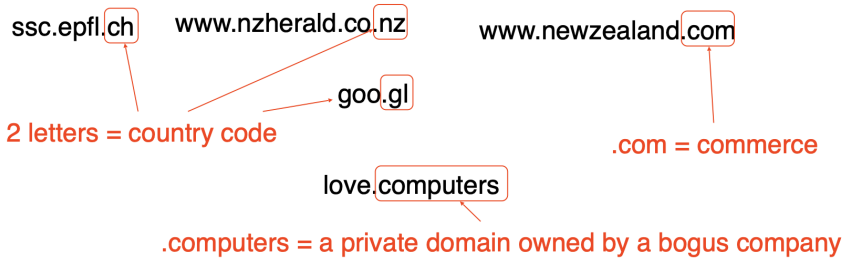


Figure 4: addresses structure examples

Application-layer example: DNS and HTTP

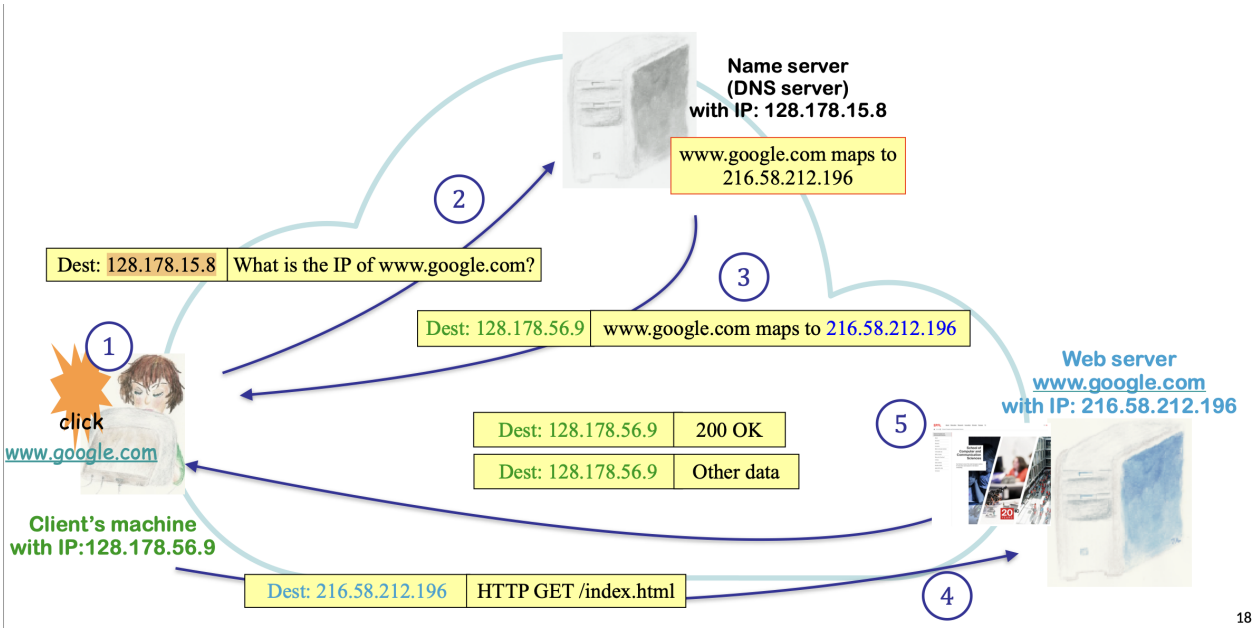


Figure 5: application layer example

Transport layer helps the application layer

Provides:

- programming interface to application layer, named socket (= specific software structure)
- (along with all layers below) data delivery to the applications of the communicating host

Exists mainly in two* versions:

TCP (Transmission Control Protocol)

- Reliable: if some data is lost somewhere, TCP retransmits it
- In-order delivery (stream service): data is delivered at destination in the order sent by source
- Unit of information is a byte-data is written to the socket, TCP sends blocks of data at will

UDP (User Datagram Protocol)

- Unreliable: message may be lost
- No in-order delivery guaranteed: data may arrive at destination out of order
- Unit of information is a message-blocks of data are sent as written into the socket

*we will also study QUIC, which is a kind of "super-TCP" and is over UDP

Examples:

TCP: video streaming as example. Needs to keep state about the connection, it's heavier.

UDP: DNS, gaming (we don't care bout losses so much), train timetable.

Transport Layer uses port numbers

- A port number is assigned to a process to uniquely identify it within a host machine
- Using ports, multiple network programs may run on a host at the same time
- A UDP/TCP packet carries both the source and destination port numbers (at its header)
- Some port numbers are reserved (e.g., 80 is for HTTP Web servers, 53 is for DNS servers, 443 is for HTTPS servers, etc.)

Network Layer provides interconnection among all network devices



Nowadays: the Internet uses Packet Switching

- data is broken into chunks called packets of size $\leq$ 1500 bytes
- packets are forwarded from one intermediate node (a.k.a. router) to another, until they reach destination. Packets may follow different paths
- similar to a Postal Service that sends only postcards:
  - Each packet $\approx$ postcard
  - is treated/delivered separately,
  - contains information related to its delivery (e.g. destination address)
  - A packet starts being forwarded on a link after its last bit has been received and all the bits of the previous packets have left.
  - Routers forward packets separately to other routers based on routing roules

Network layer: 2 basic functions

Forwarding

- When a packet arrives at a router's input link, the router moves the packet to the appropriate output link
- based on information carried in the packet's header

Routing

- The network layer determines the route/path taken by packets, as they flow from a source to a destination
- The algorithms that calculate these paths are called routing algorithms

Most important protocol: IP (Internet protocol

- interconnects multiple local area networks (LANs)
- uses packet switching; delivers packets from a source to a destination via a series of routers
- forwards packets from router to router based on IP addresses
- offers no reliable-delivery guarantees (best-effort approach)
  - packets are briefly stored in router's buffer
  - packets of the same source-destination flow may follow different routes/paths
  - so, packets may be dropped, delayed or reordered

This is why UDP is not reliable: because of the non-reliability of IP.

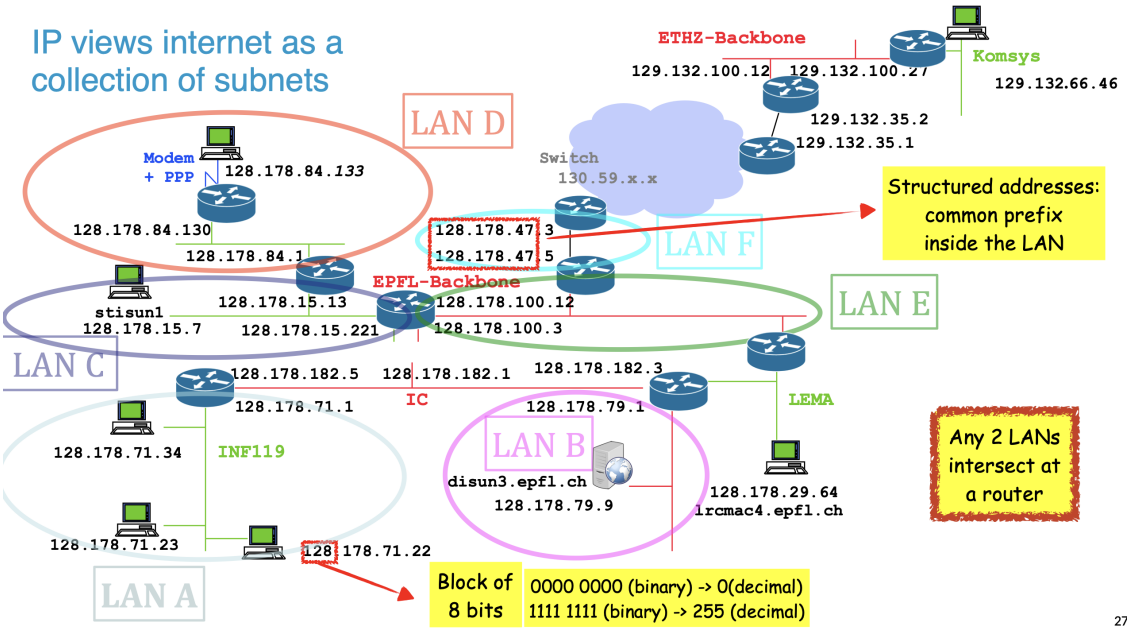


Figure 6: Routers are only placed in the edge of a subnet.

Two IP versions

IPv4 addresses have:

- a length of 32 bits
- dotted decimal notation - one number in $\{0, 1, ..., 254, 255\}$ = 8 bits
  - an EPFL (public) address: 128.178.156.23

IPv6 addresses have:

- a length of 128 bits
- hexadecimal notation - one hexadecimal in $\{0, 1, ..., e, f\}$ = 4 bits
  - an EPFL (public) address: 2001:0620:0618:01a6:0a00:20ff:fe78:30f9

IPv4 and IPv6 are incompatible

- IPv6 is another protocol (not IPv4 with a larger address space)
- devices that run IPv6 cannot parse IPv4 packets and vice versa

Why IPv4 and IPv6 ? Because IPv4 IP address space has already been saturated, Only 2 to the power of 32 address space.

Apart from addresses, IP also uses subnet masks

- subnet $\Leftarrow$ a LAN, i.e. a set of devices:
  - connected at the Data-link layer
  - sharing the same IP-address prefix, e.g.: 128.178.71.X
- The prefix is specified using a subnet mask
  - e.g. for an EPFL IPv4 LAN, the subnet mask is 1111 1111 1111 1111 1111 1111 0000 0000
  - The sequences of 1s is contiguous
  - The size (in bits) of the prefix is not always the same. ETHZ IPv4 LANs = 26 bits, EPFL IPv6 LANs = 64 bits
- Different notations for the subnet mask:
  - dotted, decimal: e.g., address = 128.178.71.34, mask = 255.255.255.0
  - "/" (slash): e.g. 128.178.71.34/24 or 2001:620:618:1a6:0a00:20ff:fe78:30f9/64

Data link layer delivers data from hop to hop

a.k.a. MAC (= Medium Access Control) layer

Responsible for:

- moving data from a node to its neighbour over a shared medium/link (wired or wireless)

- managing the access to the physical link
- providing various services to the network layer depending on the actual physical medium

Possible services:

- error detection and correction
- reliable delivery
- collision avoidance
- flow control
- link access and multiplexing of the shared medium/link

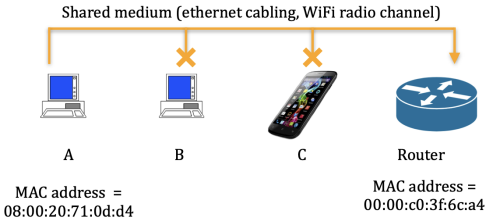
Data link layer uses MAC addresses

- MAC address (≈ device's serial no):
  - 48 bits
  - set by manufacturer
  - unique, in principle
  - no special structure



How things work?

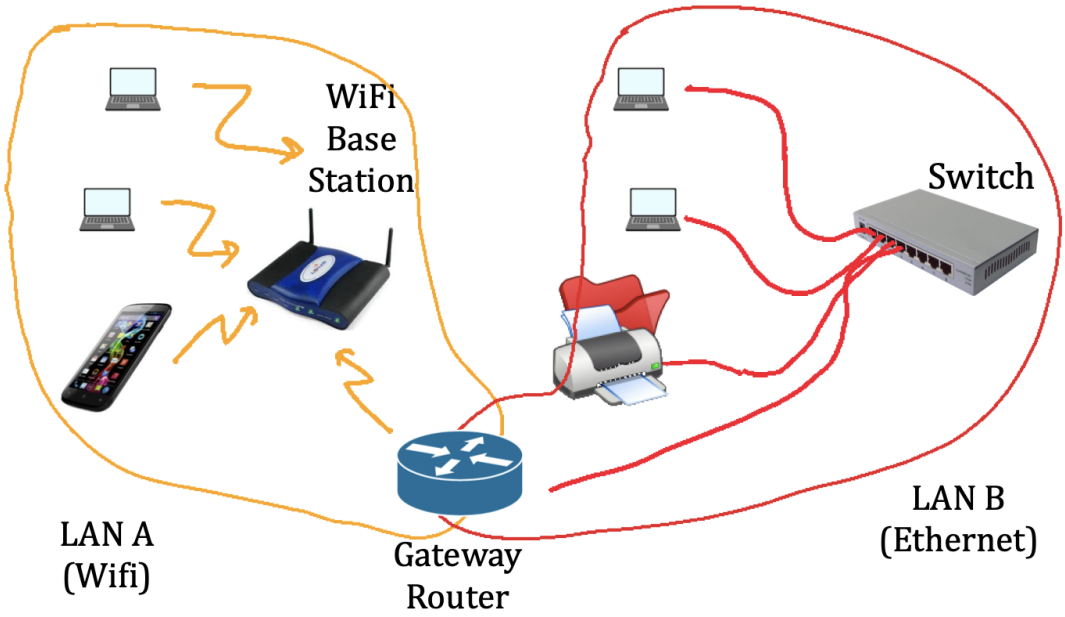
- Sender puts destination MAC address in header of the packet (called frame)
  - Destination MAC address is sent in clear-text, no encryption (but data may be encrypted)
- All nodes connected via the same/shared medium read all frames; keep only if destination MAC address matches their own
- If physical links are too long or too many, frames are stored and forwarded (as in IP) via switches



Router vs Switches

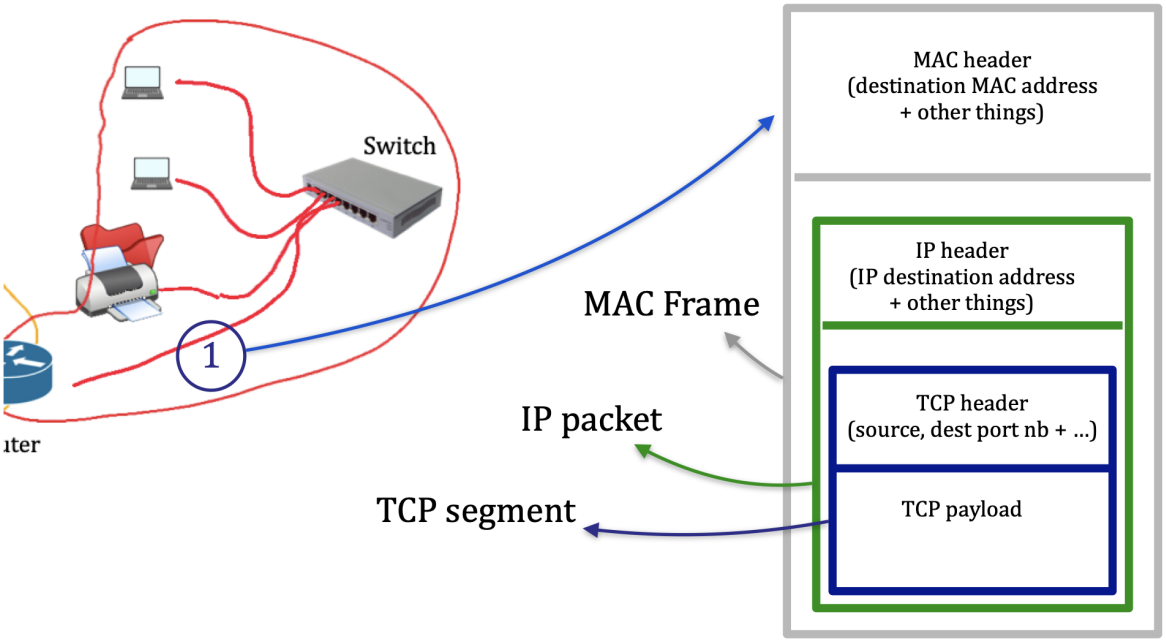
Router: a network-layer system (or program) that forwards packets based on IP addresses

Switch: a data-link-layer system (or program) that forwards frames based on MAC addresses

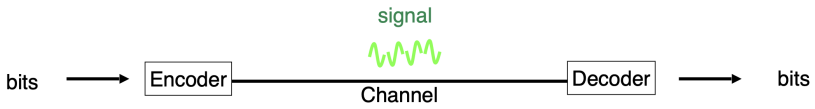


- Routers interconnect different LANs/subnets at the network layer
- Switches interconnect devices of the same LAN at the data-link layer
- To send packets outside our subnet, we send them to the LAN's first-hop router (= the gateway)
- To send packets within out subnet, we do not use any router, only switches.

Encapsulation: an onion view with headers and payloads



Physical Layer transforms Bits and Bytes into signals



- Signals are electromagnetic or acoustic (underwater)
- Technology-specific layer:
  - various Ethernet physical layer, various WLAN 802.11 physical layers
  - with various bit rates, measured in b/s, 1kb/s = 1000 b/s, 1Mb/s = $10^6 b/s, 1Gb/s = 10^9 b/s$

Bit Rate vs Bandwidth - for us they will be the same, but...

Bit rate = number of bits that can be transmitted per sec inside a channel

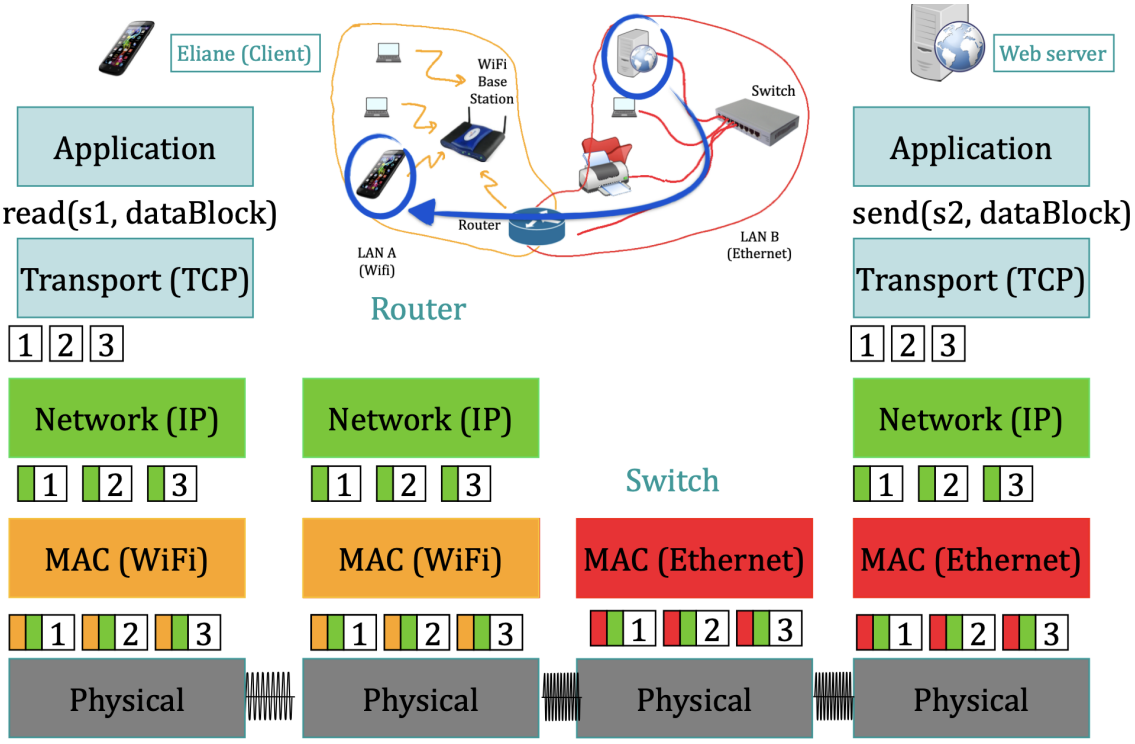
Bandwidth = the width of the frequency range that can be used for transmission over a channel

- The bandwidth limits the capacity (= max achievable bit rate) of a channel
- Information Theory computes the capacities of various channels:

- e.g. [Shannon-Hartley law], for a channel with bandwidth **B** (Hz) submitted to Gaussian noise, the capacity in b/s is $C = B \log_2(1 + SNR)$
- in practice, for a ADSL line: $B = 1MHz$, $SNR = 45dB$, $C = 15Mb/s$

Putting everything together

Suppose the web server send a file to Eliane's smarthphone



Terminology

The green packets from the network are called **IP packet**

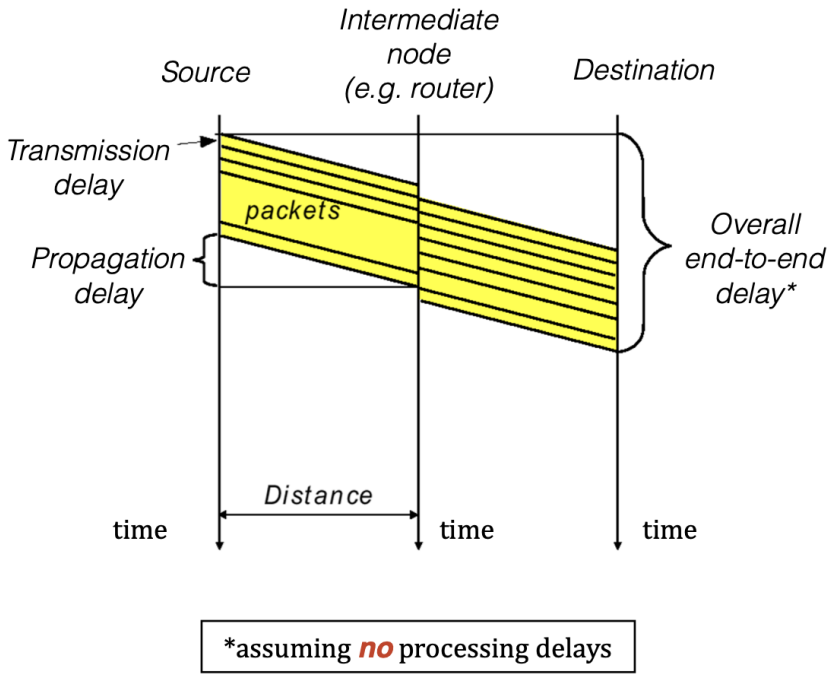
The white packets from transport are called **TCP segment**

The orange and green packets from MAC are called **MAC frame**

Network delays

- **Processing delay**: time required to examine a packet's IP header and determine where to forward it
- **Queueing delay**: time a packet needs to wait at a queue until it is transmitted into the outgoing link
- **Transmission delay** $= \frac{packetSize}{bitRate}$: time needed to push all of the packet's bits into the outgoing link

- **Propagation delay**: time for a bit to propagate from the beginning of the link to the other end, with the speed of light



Throughput

Throughput $= \frac{usefulData}{timeUnit}$, with same units as the bit rate.

Even if computed over a single link, it is not the same as the bit rate. Why?

- overhead: all protocols use some extra bytes to exchange metadata
- waiting times: a host at one end may wait a response from the other end before transmitting